

## 18. Mitigation

To mitigate against **Clickjacking** attacks, developers must configure their web servers or load balancers to include `X-Frame-Options` or `Content-Security-Policy` header.

Both `X-Frame-Options` and `Content-Security-Policy` response headers define whether or not a browser should be allowed to embed or render a page in an `<iframe>` element. For example, setting `X-Frame-Options: deny` will prevent browsers from rendering your web application in an `<iframe>` element.

Let's see how the above best practices can be applied to a sample web server configuration.

Instructions



Finish &gt;&gt;



CODE

```
1  # Enable on Nginx
2  add_header X-Frame-Options "sameorigin" always;
3  # Enable on Apache
4  header always set X-Frame-Options "sameorigin"
5  # Content Security Policy
6  Content-Security-Policy: frame-src https://www.coinpay.com/
```

## ← Command Injection

13/13 ▾

### 13. Mitigation

To effectively mitigate against **Command Injection** attacks, developers must avoid passing **user-controllable input** in functions or system calls that interface with the operating system environment or invoke third-party applications.

If this is unavoidable, development teams must perform rigorous input validation against all **user-supplied data** which includes:

- Validating against an allowlist of permitted values.
- Validating the input length.
- Validating that the input contains only alphanumeric values, ignoring all other escape characters or whitespace string.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Command Injection** attacks.

Instructions



Finish &gt;&gt;

 CODE

```

10     response.getWriter().append("Submitted phone number is not
    valid");
11     return;
12 }
13
14     Runtime runtime = Runtime.getRuntime();
15
16     try {
17         Process process = runtime.exec(new String[] {
18             "bash",
19             "-c",
20             "curl -o -I -L -s -w %{http_code}" + API_URL + "?number=" +
phoneNumber
21             // curl -o -l -L -s -w %{http_code}
https://apigree.sms.gateway.com?number=
22         });
23
24         String processResponse = readResponseFor(process);
25         response.getWriter().append(processResponse);
26     } catch (InterruptedException | IOException e) {
27         LOGGER.error("Error while processing the request: ", e);
28         response.getWriter().append(e.getMessage());

```

## ← Forced Browsing

15/15

### 15. Mitigation

In order to mitigate against **Forceful Browsing** vulnerabilities, developers must ensure appropriate permissions and access-control settings are applied for every **URL** and **web resource** in their application.

Further, application owners must remove all private URL routes used for **development** and **testing** in production environments.

For example, if an application has a private URL route for accessing an internal-only web interface or API, developers must scope the private URL routes so that only users with the appropriate access rights can access these interfaces.

Finally, developers must protect application

Instructions



When you are ready, click **NEXT** to complete the exercise.

Finish &gt;&gt;

https://41.208.179.105/user/dashboard/monitor





Bob

Zebra MQ Console

 Search

| Status | Client              | Check                          | Output                                                                                                                                                          |
|--------|---------------------|--------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| warn   | sensu-backend-o-vpc | autoscale_vxcode_healthy_hosts | autoscale healthy hosts WARNING:vxcode-asg-c3 has 0 healthy hosts                                                                                               |
| crit   | i-e713f288          | keepalive                      | no keep-alive sent from client in over 180 seconds                                                                                                              |
| crit   | i-e71e2cd1e         | chef-process                   | PROCS CRITICAL: 0 processes with command name 'client'                                                                                                          |
| crit   | i-of0b3e7f          | ntpd                           | PROCS CRITICAL: 0 processes with command name 'ntpd'                                                                                                            |
| crit   | i-of0b3e7f          | rsyslogd                       | PROCS CRITICAL: 0 processes with command name 'rsyslogd'                                                                                                        |
| warn   | i-4b99233a          | check_disc                     | /etc/sensu/plugins/check-disc.rb:15 in 'require':cannot load such file -- sensu-plugin/check/cli(LoadError)from /etc/sensu/plugins/check-disc.rb:15:in '<main>' |
| warn   | i-4b99233a          | check_dns                      | /etc/sensu/plugins/check-dns.rb:23:in 'require':cannot load such file -- sensu-plugin/check/cli(LoadError)from /etc/sensu/plugins/check-dns.rb:23:in '<main>'   |
| warn   | i-                  | check_load                     | /etc/sensu/plugins/check-load.rb:12:in 'require':cannot load such file -- sensu-                                                                                |



## 19. Mitigation

To mitigate the risk of malicious actors exploiting **URL redirects** in the way that **Bob** did, application developers must always verify the URL that the user will be **redirected** or **forwarded** to, especially if the URL parameter can be changed or tampered with on the client-side. This is especially important for applications that use dynamic content, such as web pages, that are not hosted by a single domain.

However, in our example, we can eliminate the need to perform URL verification by performing a **server-side hostname lookup** to securely redirect the user to the login webpage.

Let's see how this best practice can be applied to our **vulnerable code** example.

Instructions



Finish &gt;&gt;

&lt;/&gt; CODE

```

1  private void logout(HttpServletRequest request, HttpServletResponse
    response) throws IOException {
2      HttpSession session = request.getSession(false);
3
4      if (session != null) {
5          session.invalidate();
6      }
7
8      StringBuilder redirectUrlBuilder = new StringBuilder()
9          .append(request.getScheme())
10         .append("://")
11         .append(InetAddress.getLoopbackAddress().getHostName())
12         .append(request.getContextPath());
13
14     response.sendRedirect(redirectUrlBuilder.toString());
15 }

```

←- Leftover Debug Code

11/11 ▾

## 11. Mitigation

In order to mitigate against deploying active **debugging code in production** instances, developers must ensure appropriate permissions and access-control settings are applied for accessing such features. Alternatively, development teams can add conditional variables to remove **debugging code** when deploying in **production** and **staging** environments.

Further, development teams must proactively identify and remove developer comments that reference sensitive information including internal URLs, API endpoints, or test credentials before deploying the application.

Finally, developers must protect application **staging** and **test** environments by restricting their access to specific IP ranges or addresses, thereby preventing search engines such as

Instructions



Analyze Code

Finish >>

 CODE

```
1  private void authenticate(HttpServletRequest request) {
2      String userName = request.getParameter("username");
3      String password = request.getParameter("password");
4
5      int isHeadless =
        Integer.parseInt(request.getParameter("headless"));
6      HttpSession session = request.getSession(true);
7
8      Boolean isProduction = SystemProperty.environment.value() ==
        SystemProperty.Environment.Value.Production;
9
10     // For debugging and Selenium tests only. Remove later.
11     if (isHeadless == 1 && !isProduction) {
12         session.setAttribute("isAdmin", true);
13         loginSelenium(session);
14     } else {
15         login(userName, password, session);
16     }
17 }
```

## ← Reflected XSS

19/19 ▾

### 19. Mitigation

In order to effectively mitigate against **Cross-site Scripting** attacks, developers must ensure all user-supplied data is **HTML encoded** before being rendered to the web page.

**HTML encoding** ensures an application will always convert reserved HTML characters such as `&`, `<`, `>`, `"`, `'`, `/` into their equivalent HTML entities.

For example, the input value `<script>` will be converted to `&lt;script&gt;`, ensuring that the browser renders the output as data and not HTML markup.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Cross-Site Scripting** attacks.

Instructions



Finish &gt;&gt;



CODE

```

1  <%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
2  <html>
3  <body>
4
5  <form name="search" method="GET">
6      <input name="searchParameter"/>
7  </form>
8
9  <c:if test="${resultSetEmpty}">
10     No results found for: <c:out value="${searchParameter}"
11     escapeXml="true"/>
12 </c:if>
13
14 <c:forEach items="${foundIssues}" var="issue">
15     <tr>
16         <td>${issue.id}</td>
17         <td>${issue.title}</td>
18         <td>${issue.description}</td>
19         <td>${issue.status}</td>
20     </tr>
21 </c:forEach>

```



## 19. Mitigation

In order to effectively mitigate against **SQL Injection** attacks, developers must use **prepared statements** (also known as **parameterized queries**) when building SQL queries based on user input.

**Prepared statements** prevent SQL Injection attacks by defining **placeholder** variables to safely pass **parameters** inside a SQL statement, which are automatically escaped at runtime.

Let's see how a **prepared statement** can be applied to our vulnerable code example to remediate the **SQL Injection** vulnerability.

Instructions



Finish &gt;&gt;

CODE

```
1  private void unsubscribeUser(HttpServletRequest request,
    HttpServletResponse response) throws ServletException, IOException
    {
2      JSONObject json = new JSONObject();
3      String email = request.getParameter("email");
4
5      try {
6          Connection db = DBHelper.getConnection();
7          String sql = "SELECT email FROM user WHERE email = ?";
8
9          PreparedStatement prepareStatement = db.prepareStatement(sql);
10         prepareStatement.setString(1, email);
11
12         ResultSet results = prepareStatement.executeQuery(sql);
13
14         if (results.next()) {
15             email = DBHelper.getEmailColumn(results);
16             DBHelper.UnsubscribeUser(email);
17         }
18
19         json.put("email", email);
20         json.put("success", true);
```

## ← Stored Cross Site Scripting

15/15 ▾

### 15. Mitigation

In order to effectively mitigate against stored **Cross-site Scripting** attacks, developers must ensure all user-supplied data is **HTML encoded** before being rendered to the web page.

**HTML encoding** ensures an application will always convert reserved HTML characters such as `&` `<` `>` `"` `'` `/` into their equivalent HTML entities.

For example, the input value `<script>` will be converted to `&lt;script&gt;`, ensuring that the browser renders the output as data and not HTML markup.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **Cross-Site Scripting** attacks.

Instructions



Finish &gt;&gt;

&lt;/&gt; CODE

```

3  <body>
4
5  <div class="details align-left">
6    <p class="bold-text divider">Details</p>
7    <p class="details-item">Type: <c:out value="${issue.type}"/></p>
8    <p class="details-item">Status: <c:out value="${issue.status}"/>
9    <p class="details-item">Priority: <c:out
10   value="${issue.priority}"/></p>
11   <p class="details-item">Resolution: <c:out
12   value="${issue.resolution}"/></p>
13 </div>
14 <div class="people align-right">
15   <p class="bold-text divider">People</p>
16   <p class="people-item">Assignee: <c:out
17   value="${issue.assignee}"/></p>
18   <p class="people-item">Reporter: <c:out
19   value="${issue.reporter}"/></p>
20 </div>
21 <div class="issue-description divider">
22   <p class="description-border">Description</p>
23   <c:out value="${issue.description}"/>
24   <!-- &amp;lt;script&amp;gt;ot:new

```



## 8. Mitigation

Passing sensitive data such as **API tokens** within **URLs** may be logged in various locations, including the user's browser, the web server, and any forward or reverse proxy servers. Application **URLs** may also be posted on public forums, bookmarked or emailed by developers, thereby increasing the risk of invariably disclosing **API tokens**.

To effectively mitigate against exposure of API tokens in URLs, developers should use **HTTP** headers for transmitting **API tokens**. Examples of these headers include the following:

- **Authorization:** <token\_value>
- **X-API-Key:** <token\_value>
- **ProjectName-API-Key:** <token\_value>

Instructions



Click **NEXT** to complete the exercise.

Finish >>

```
{
  {
    "id": 1,
    "description": "Internal Tradetech charting library for Terminal Desk",
    "name": "Flight",
    "name_with_namespace": "Tradetech Chart Library v1",
    "path": "flight",
    "path_with_namespace": "tradetech/chart_lib",
    "created_at": "2017-09-05T07:58:01.621Z",
    "default_branch": "master",
    "tag_list": [],
    "ssh_url_to_repo": "ssh://alice@192.168.33.13:2222/tradetech/chart_lib.git",
    "http_url_to_repo": "http://192.168.33.13:3000/tradetech/chart_lib.git",
    "web_url": "http://192.168.33.13:3000/tradetech/chart_lib",
    "avatar_url": null,
    "star_count": 2,
    "forks_count": 5,
    "last_activity_at": "2018-01-31T09:56:30.902Z"
  }
}
```

CONSOLE

```
alice@www.codedock.com:~$ tail -f /var/www/access.log

192.168.1.110 - - [26/Aug/2019:17:06:41 +1000] "GET / HTTP/1.0" 200 3488
 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
 like Gecko) Chrome/60.0 33112 Safari/537.36"

211.1.17.135 - - [26/Aug/2019:18:09:27 +1000] "GET /api/v4/projects?
 private_token=t_zPRYZ4pMRQXyznF72g" 200 3488 "-" "Mozilla/5.0 (Windows NT
 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0
 33112 Safari/537.36"
```

## 13. Mitigation

To effectively mitigate against **User Enumeration** attacks, developers must display a **generic response message** regardless of whether or not the username, email or account is valid.

Let's see how the above best practice can be applied to our vulnerable code example to prevent **User Enumeration** attacks.

Instructions



Finish &gt;&gt;



CODE

```

1  private void resetPassword(HttpServletRequest request,
    HttpServletResponse response) {
2      String email = request.getParameter("email");
3      String message = "Please check your email inbox to continue";
4      User user = DBHelper.findUserByEmail(email);
5
6      if (user == null) {
7          forwardUserToLogin(request, "login", message);
8          return;
9      }
10
11     String newPassword = PasswordUtil.generateRandomPassword();
12     StringBuilder hostBuilder = new StringBuilder()
13         .append(request.getScheme())
14         .append("://")
15         .append(InetAddress.getLoopbackAddress().getHostName())
16         .append(request.getContextPath())
17         .append("/resetpwd?pwd=")
18         .append(newPassword);
19
20     DBHelper.updateUserDetails(email, newPassword);
21     MailHelper.sendMail("admin", email, hostBuilder.toString());

```



## Vertical Privilege Escalation

14/14

### 14. Mitigation

To effectively mitigate against **Privilege Escalation** vulnerabilities, developers must implement access control checks to make sure that the user has the required privileges to access a requested resource or functionality.

A proper access control policy is important on the whole product level. It should define appropriate access control permissions for all users and groups. These then need to be described clearly to architects, designers, developers and support teams, such that access control is designed, implemented and used consistently across the application.

As a first step, let's see how we can include a simple access control check in our vulnerable code example to prevent **Privilege Escalation** attacks.

Instructions



Finish &gt;&gt;

CODE

```

1  public void changePassword(HttpServletRequest request,
    HttpServletResponse response) throws IOException {
2      HttpSession session = request.getSession();
3
4      if (session != null) {
5          User user = session.getAttribute("user");
6          String newPassword = request.getParameter("newPassword");
7          String oldPassword = request.getParameter("oldPassword");
8          String account = request.getParameter("account");
9
10         user.setPassword(newPassword);
11         user.setOldPassword(oldPassword);
12
13         if (user.getAccountId().equals(account)) {
14             updatePasswordFor(user);
15
16             response.setContentType("application/json");
17             PrintWriter pw = response.getWriter();
18             pw.println(buildJSONResponse(user));
19             pw.flush();
20         }
21     }

```



## 11. Mitigation

In order to effectively mitigate against **XXE injection** attacks, developers must configure their applications **XML parsers** to disable the parsing of **XML eXternal Entities** (XXE) and **Document Type Definitions** (DTD) when parsing XML documents.

If **DTDs** cannot be completely disabled, developers must disable the parsing of external general entities and external parameter entities when parsing untrusted XML files.

Let's see how the above best practices can be applied to our vulnerable code example to prevent **XML injection** attacks.

Instructions



Finish &gt;&gt;



CODE

```

1  public class PeakFitnessBuilderFactory {
2      private DocumentBuilderFactory documentBuilderFactory() {
3          DocumentBuilderFactory documentBuilderFactory =
4              DocumentBuilderFactory.newInstance();
5
6          try {
7
8              documentBuilderFactory.setFeature("http://apache.org/xml/features/
9                  disallow-doctype-decl", true);
10
11              documentBuilderFactory.setFeature("http://xml.org/sax/features/ext
12                  ernal-general-entities", false);
13
14              documentBuilderFactory.setFeature("http://xml.org/sax/features/ext
15                  ernal-parameter-entities", false);
16          } catch (ParserConfigurationException e) {
17              throw new RuntimeException(e);
18          }
19
20          return documentBuilderFactory;
21      }
22  }

```